

Model-View-Controller

Na web

Em 2002

```
<?php
echo $title;
?>
<style>
table {
    font-weight: bold;
}
</style>
<?php

echo '<h1>New Users</h1>';

$sql = "SELECT * FROM users ORDER BY date_registered";
$result = mysql_query($sql) or die(mysql_error());

echo '<table class="my-table-class">';
while($row = mysql_fetch_assoc($result)){
    echo '<tr><td>' . $row['username'] . '</td><td>' . $row['date_registered'] . '</td></tr>';
}
echo '</table>';

function random_custom_function($var){
    $var = $var + 1;
    return '<span style="font-weight:bold;">' . $var . '</span>';
}

$sql = "SELECT * FROM table WHERE column = 'test'";
$result = mysql_query($sql) or die(mysql_error());

echo '<div id="test">';
$i = 0;
while($row = mysql_fetch_assoc($result)){
    if($row['type'] == 3){
        echo '<div style="margin-bottom:20px;">' . random_custom_function($row['val']) . '</div>';
        $i++;
    }
    else{
        echo '<div style="margin-bottom:20px;">' . $row['val'] . '</div>';
    }
}

if($i == 0){
    echo '<table>';
    echo '<tr><td>Found none!</td></tr>';
    echo '</table>';
}
```

Em 2002

```
<?php
echo $title;
?>
<style>
table {
    font-weight: bold;
}
</style>
<?php

echo '<h1>New Users</h1>';

$sql = "SELECT * FROM users ORDER BY";
$result = mysql_query($sql) or die(my

echo '<table class="my-table-class">';
while($row = mysql_fetch_assoc($result))
    echo '<tr><td>' . $row['username']
}
echo '</table>';

function random_custom_function($var)
    $var = $var + 1;
    return '<span style="font-weight:

}

$sql = "SELECT * FROM table WHERE col";
$result = mysql_query($sql) or die(my

echo '<div id="test">';
$i = 0;
while($row = mysql_fetch_assoc($result))
    if($row['type'] == 3){
        echo '<div style="margin-bott
        $i++;
    }
    else{
        echo '<div style="margin-bott
    }
}

if($i == 0){
    echo '<table>';
    echo '<tr><td>Found none!</td></tr>';
    echo '</table>';
}
```

- Problema:
 - Código de diferentes aspectos misturados!
 - PHP (Lógica de negócio)
 - SQL (Acesso a dados)
 - HTML (Estrutura da Apresentação)
 - CSS (Estilo da Apresentação)
 - Javascript (Lógica da Apresentação)
 - Ou os webdesigners precisam de perceber tudo!
 - Podem facilmente estragar a lógica da aplicação.

WebObjects, NeXT (Apple)

MVC na Web

- MVC, mas adaptado à web!
- NeXT (hoje, parte da Apple) lançou o primeiro MVC para a web:
 - **WebObjects**

WebObjects, NeXT (Apple)

MVC na Web

Model objects represent special knowledge and expertise. For example, **model objects can hold a company's data and define the logic that manipulates that data.** Model objects are not directly displayed. They often are reusable, distributed, persistent, and portable to a variety of platforms.

View objects represent things visible on the user interface (windows, for example, or buttons). A View object is “ignorant” of the data it displays. View objects tend to be very reusable and so provide consistency between applications.

Acting as a mediator between Model objects and View objects in an application is a Controller object. There is usually one per application or window. **A Controller object communicates data back and forth between the Model objects and the View objects.** Since what a Controller does is very specific to an application, it is generally not reusable even though it often comprises much of an application's code.

Because of the Controller's central mediating role, Model objects need not know about the state and events of the user interface, and View objects need not know about the programmatic interfaces of the Model objects.

Ruby on Rails

MVC na Web

```
app > models > article.rb
1  # == Schema Information
2  #
3  # Table name: articles
4  #
5  #  id      :integer      not null, primary key
6  #  title   :string
7  #  text    :text
8  #  created_at :datetime    not null
9  #  updated_at :datetime    not null
10 #
11 class Article < ApplicationRecord
12   belongs_to :user
13   has_many :comments, dependent: :destroy
14   validates :text, presence: true
15   before_validation :normalize_name, on: :create
16 end
17
18
```

Models - representa linhas da base de dados, inclui lógica de negócio.

Ruby on Rails

MVC na Web

```
app > controllers > articles_controller.rb
1  class ArticlesController < ApplicationController
2    before_action :authenticate_user!, except: [:index]
3
4    def index
5      @articles = Article.all
6    end
7
8    def show
9      @article = Article.find(params[:id])
10   end
11
12   def new
13     @article = Article.new
14   end
15
16   def edit
17     @article = Article.find(params[:id])
18   end
19
20   def create
21     @article = Article.new(article_params)
22     @article.user = current_user
23     if @article.save
24       redirect_to @article
25     else
26       render 'new'
27     end
28   end
29
```

Controllers - manipula a lógica da camada de apresentação
(entre o html/javascript e a lógica de negócio no modelo)

Ruby on Rails

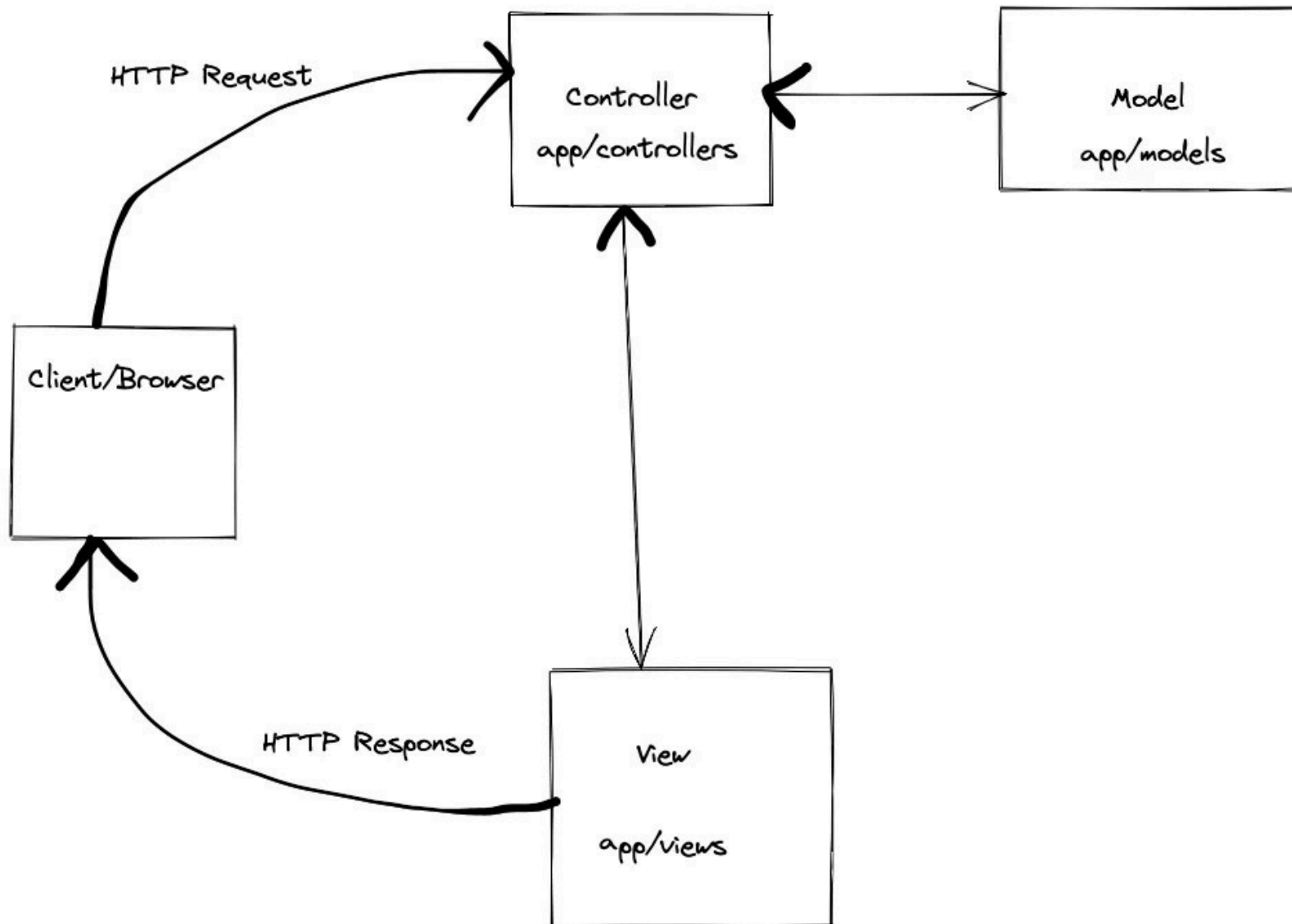
MVC na Web

```
app > views > articles > <> _list.html.erb
1  <%= stylesheet_link_tag 'style.css' %>
2  <div class="container" %>
3  <p>
4  <h4>Listing Articles</h4><a class="btn btn-info" style="margin:15px;"<%= link_to 'New Article', new_article_path %></a>
5  </p>
6  <table class="table table-bordered">
7    <tr>
8      <th>Title</th>
9      <th>Text</th>
10     <th>See Articles</th>
11     <th>Edit Articles</th>
12     <th>Destroy Articles</th>
13   </tr>
14   <% @articles.each do |article| %>
15     <tr>
16       <td><%= article.title %></td>
17       <td><%= article.text.slice(0,50)+ '...' %></td>
18       <td><%= link_to ('<i class="fa fa-eye" aria-hidden="true"></i>').html_safe, article_path(article) %></td>
19       <td><%= link_to ('<i class="fas fa-pencil-alt" aria-hidden="true"></i>').html_safe, edit_article_path(article) %></td>
20       <td><%= link_to ('<i class="fas fa-trash-alt" aria-hidden="true"></i>').html_safe, article_path(article),
21         method: :delete,
22         data: { confirm: 'Are you sure?' } %></td>
23     </tr>
24   <% end %>
25 </table>
26 |
```

Views - Ficheiros HTML estáticos com o layout das páginas

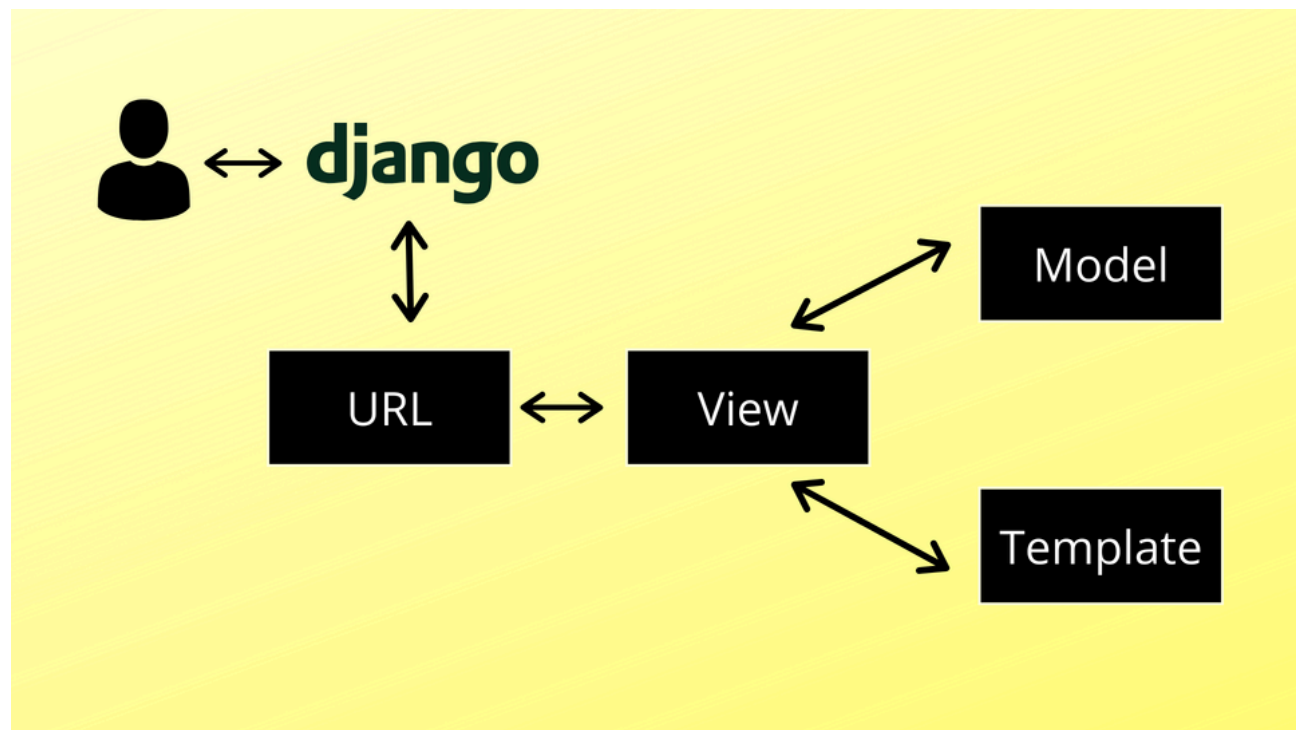
Ruby on Rails

MVC na Web



Django

MVC na Web



View - controla a resposta ao pedido HTTP

Model - Tabelas e Instâncias a que recorre para responder ao pedido

Template - HTML base, que vais ser substituído com dados pela vista.

Spring MVC

MVC na Web

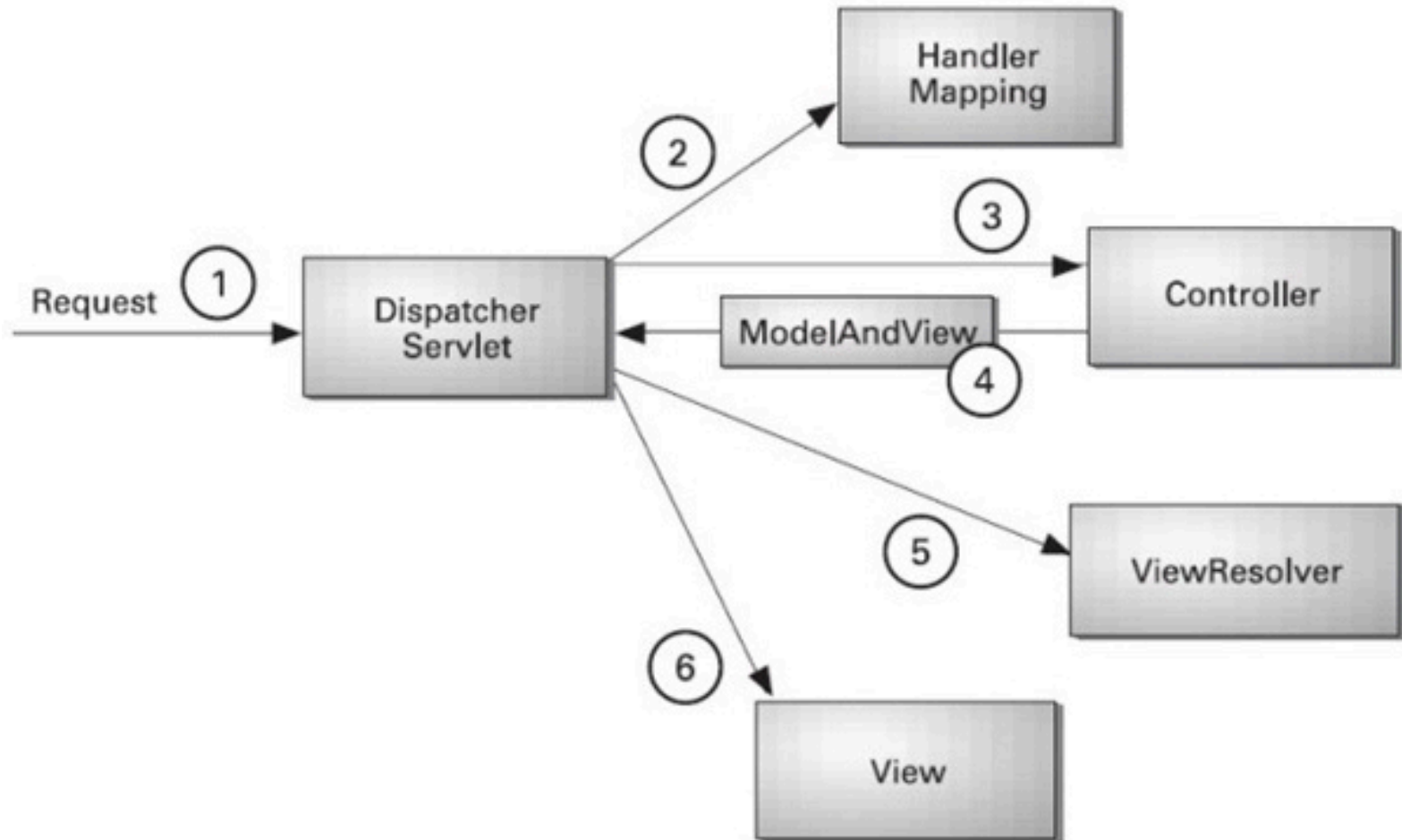
Controllers: Classes que tratam da interação do cliente (i.e., pedidos HTTP).

Views: Ficheiros HTML (Thymeleaf) que vão ser preenchidos pelos controllers.

ViewModels: Objectos POJO que são preenchidos pelos controllers e lidos pelas Views.

Spring MVC

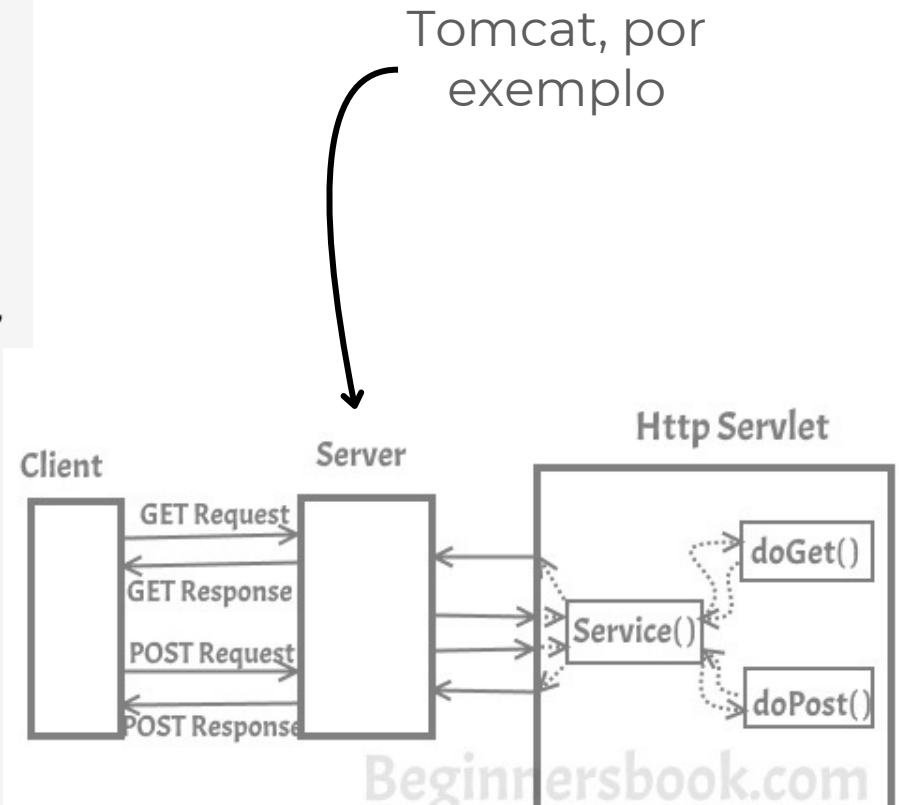
MVC na Web



Servlet

Uma classe que responde a pedidos HTTP

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
// Creating Http Servlet by Extending HttpServlet class
public class ExampleHttpServlet extends HttpServlet
{
    private String mymsg;
    public void init() throws ServletException
    {
        mymsg = "Http Servlet Demo";
    }
    public void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException
    {
        // Setting up the content type of web page
        response.setContentType("text/html");
        // Writing the message on the web page
        PrintWriter out = response.getWriter();
        out.println("<h1>" + mymsg + "</h1>");
        out.println("<p>" + "Hello Friends!" + "</p>");
    }
    public void destroy()
    {
        // Leaving empty. Use this if you want to perform
        //something at the end of Servlet life cycle.
    }
}
```



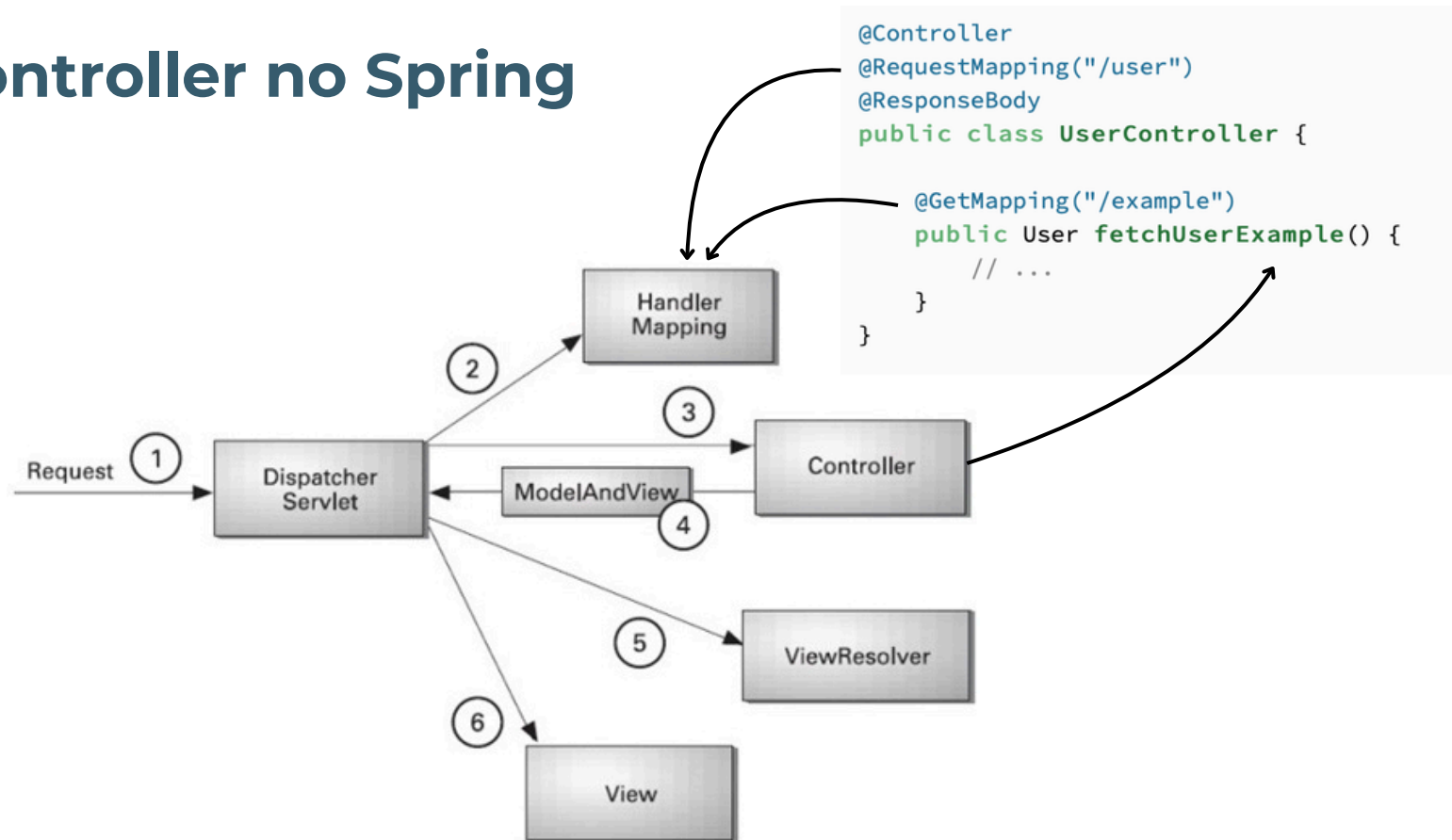
Front vs Page Controller

Padrões para aplicações Web

- Front Controller
 - 1 classe que recebe todos os pedidos, e decide a lógica para todas as páginas
 - Útil para acções comuns como autenticação, autorização, encriptação e caching
- Page Controller
 - 1 classe por página, que tem apenas a lógica dessa página
 - Útil para formulários complexos, por exemplo

DispatcherServlet

FrontController no Spring



O DispatcherServlet lida com uma HttpRequest recebida, delega a solicitação e a processa de acordo com as interfaces HandlerAdapter configuradas

Controllers

MVC no Spring

```
import org.springframework.stereotype.Controller;  
import org.springframework.ui.Model;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestParam;
```

```
@Controller
```

```
public class GreetingController {
```

```
    @GetMapping("/greeting")
```

```
    public String greeting(@RequestParam(name="name", required=false, defaultValue="World") String name,  
        Model model) {
```

```
        model.addAttribute("name", name);
```

```
        return "greeting";
```

```
    }
```

```
}
```

Controllers

MVC no Spring

```
import org.springframework.stereotype.Controller;  
import org.springframework.ui.Model;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestParam;
```

@Controller **Controllers são Components, mas indicam o propósito!**

```
public class GreetingController {  
  
    @GetMapping("/greeting")  
    public String greeting(@RequestParam(name="name", required=false, defaultValue="World") String name,  
        Model model) {  
        model.addAttribute("name", name);  
        return "greeting";  
    }  
  
}
```

Controllers

MVC no Spring

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

@Controller **Controllers são Components, mas indicam o propósito!**

```
public class GreetingController {
    @GetMapping("/greeting")
    public String greeting(@RequestParam(name="name", required=false, defaultValue="World") String name,
        Model model) {
        model.addAttribute("name", name);
        return "greeting";
    }
}
```

Mappings indicam o endereço. Get/Put/Post/Delete mappings apenas para esse verbo

Controllers

MVC no Spring

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

@Controller **Controllers são Components, mas indicam o propósito!**

```
public class GreetingController {
    @GetMapping("/greeting")
    public String greeting(@RequestParam(name="name", required=false, defaultValue="World") String name,
        Model model) {
        model.addAttribute("name", name);
        return "greeting";
    }
}
```

Mappings indicam o endereço. Get/Put/Post/Delete mappings apenas para esse verbo

@RequestParam indica que no endereço ?name=xpto, esta variável vai ter o valor xpto.

Controllers

MVC no Spring

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

@Controller **Controllers são Components, mas indicam o propósito!**

```
public class GreetingController {
    @GetMapping("/greeting")
    public String greeting(@RequestParam(name="name", required=false, defaultValue="World") String name,
        Model model) {
        model.addAttribute("name", name);
        return "greeting";
    }
}
```

Mappings indicam o endereço. Get/Put/Post/Delete mappings apenas para esse verbo

@RequestParam indica que no endereço ?name=xpto, esta variável vai ter o valor xpto.

Podem devolver HTTPResponses, Objectos (que são serializados para JSON, por exemplo) ou mais normalmente Strings com o nome da vista.

Controllers

MVC no Spring

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

@Controller **Controllers são Components, mas indicam o propósito!**

```
public class GreetingController {
    @GetMapping("/greeting")
    public String greeting(@RequestParam(name="name", required=false, defaultValue="World") String name,
        Model model) {
        model.addAttribute("name", name);
        return "greeting";
    }
}
```

Mappings indicam o endereço. Get/Put/Post/Delete mappings apenas para esse verbo

@RequestParam indica que no endereço ?name=xpto, esta variável vai ter o valor xpto.

Podem devolver HTTPResponses, Objectos (que são serializados para JSON, por exemplo) ou mais normalmente Strings com o nome da vista.

A string “greeting” vai carregar o template “greeting.html” com acesso ao objecto model.

Model

MVC no Spring

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

```
@Controller
```

```
public class GreetingController {
```

```
    @GetMapping("/greeting")
```

```
    public String greeting(@RequestParam(name="name", required=false, defaultValue="World") String name,
        Model model) {
```

```
        model.addAttribute("name", name);
        return "greeting";
```

```
    }
```

```
}
```

A interface model consiste num dicionário onde podem ser colocados beans associados a um nome. A vista depois pode aceder a estes objectos.

Views

MVC no Spring

Current time is <span
th:text="\${serverTime}" />

Tags HTML podem ter atributos que começam com o namespace “th”, de thymeleaf. Estes serão processados no servidor! Nunca chegarão a ser vistos pelo browser, nem no ver código fonte.

Nos valores destes tags, podemos usar EL - Expression language.

`${abc}` - avalia abc durante o carregamento do template.

`#{abc}` - avalia abc durante uma fase posterior do processamento do template.

`${abc.xyz}` - corresponde a `model.getAttribute(“abc”).getXYZ()`

Vista

MVC no Spring

Se `students` for um `ArrayList`, podemos usar o `th:each` para os percorrer, tal como um `foreach`.

```
<tbody>
  <tr th:each="student: ${students}">
    <td th:text="${student.id}" />
    <td th:text="${student.name}" />
  </tr>
</tbody>
```

Vista

MVC no Spring

th:if e th:unless funcionam como blocos condicionais para esse elemento aparecer.

```
<td>  
  <span th:if="${student.gender} == 'M'" th:text="Male" />  
  <span th:unless="${student.gender} == 'M'" th:text="Female" />  
</td>
```

```
<form action="#" th:action="@{/saveStudent}" th:object="${student}" method="post">
  <table border="1">
    <tr>
      <td><label th:text="#{msg.id}" /></td>
      <td><input type="number" th:field="*{id}" /></td>
    </tr>
    <tr>
      <td><label th:text="#{msg.name}" /></td>
      <td><input type="text" th:field="*{name}" /></td>
    </tr>
    <tr>
      <td><input type="submit" value="Submit" /></td>
    </tr>
  </table>
</form>
```

th:action preenche a action com o controlador a submeter. O **th:object** carrega os campos com um objecto do modelo (útil para editar entidades).

Server-Side Rendering vs Single-Page Applications

Vantagens e Desvantagens

- **SSR** é “lento”. Cada link requer um carregamento completo da página. **SPAs** parecem mais rápidas: as requisições contêm apenas novos dados, e não um novo layout (o conteúdo parcial é substituído).
 - A primeira requisição de uma **SPA** demora mais; as requisições subsequentes são mais rápidas.
- O **SSR** possui URLs individuais automaticamente. As **SPAs** podem ter isso utilizando a API de History do HTML5 (ou um framework de roteamento), o que requer esforço do desenvolvedor.
 - URLs individuais geram melhor otimização para mecanismos de busca (SEO), melhor acessibilidade, compartilhamento e arquivamento.

Model-View-Controller

Na web